

Rappresentazione dell'informazione

Definire «informazione»

- Cos'è l'informazione? Proposta:
 - informazione è un qualunque insieme di segnali che condizionano l'evoluzione di un sistema
- Consideriamo, ad esempio, il termostato di una caldaia. Lo scendere della temperatura al di sotto di una certa soglia genera un segnale di attivazione che accende la caldaia riscaldando l'ambiente. Di contro, raggiunto un altro valore prefissato della temperatura, un secondo segnale provoca lo spegnimento della caldaia.

Definire «informazione»

- Sembra una buona definizione ma
 - Prediamo ora come esempio una automobile: girando lo sterzo invio un segnale che modifica la traiettoria dell'auto facendola curvare
 - Esempi simili: ruotare la manovella di un crick modifica l'inclinazione di una macchina
- Così facendo qualsiasi cosa perturbi l'evoluzione di un sistema potrebbe essere considerato informazione
 - Problema: occorre definire cosa sia un segnale
 - Rispetto al termine informazione ci sembra più familiare e possiamo associarlo ad un gran numero di esempi concreti: segnali elettrici, caratteri alfanumerici, pittogrammi. Tuttavia, definire nella sua generalità il termine segnale potrebbe essere non meno difficile di definire cosa sia un'informazione
 - Occorre poi stare attenti a non cadere in definizioni circolari: un segnale è un'azione o evento che veicola una qualche informazione

Definire «informazione»

- Altri esempi/dubbi
 - Supponete di stare in macchina diretti verso il porto. Ad un certo punto dite al conducente che per il porto occorre svoltare a destra ma il conducente vi ignora e prosegue. Poiché quanto avete detto non ha condizionato l'evoluzione del sistema (la macchina ha continuato a tirare dritto) allora non può essere considerato un'informazione.
- Sembra che il concetto di informazione quindi non dipenda necessariamente dalla effettiva evoluzione di un sistema, né che sia indissolubilmente legato al concetto di segnale.
- Allora la definizione è sbagliata?
 - In realtà se andate a cercare ad esempio sul vocabolario Treccani non ne troverete una migliore
 - Il problema è che la nozione di informazione, per la sua generalità, sembra essere una nozione primitiva
 - Non esiste un'unica definizione soddisfacente, ma ci si può accontentare di diverse nozioni, ognuna delle quali evidenzia un particolare aspetto
 - Un'alternativa è quella di restringere il contesto in cui si intende una certa nozione per darne una definizione più precisa

Alfabeti, parole, linguaggi

- Per alfabeto A intenderemo un insieme finito e distinguibile di segni che chiameremo a seconda del contesto cifre, lettere, caratteri, simboli etc.
 - Le nove cifre dell'alfabeto decimale $A=\{'0',\dots,'9'\}$
 - Le ventisei lettere (minuscole) dell'alfabeto $A=\{'a',\dots,'z'\}$
 - I quattro simboli delle carte francesi $A=\{\clubsuit, \diamondsuit, \heartsuit, \spadesuit\}$
- Una parola (o stringa) su A è una sequenza finita di simboli dell'alfabeto A
 - “18945” ($A=\{'0',\dots,'9'\}$)
 - “pkwocod” ($A=\{'a',\dots,'z'\}$)
 - “ $\clubsuit \clubsuit \diamondsuit \spadesuit$ ” ($A=\{\clubsuit, \diamondsuit, \heartsuit, \spadesuit\}$)
- Con A^* indicheremo tutte le possibili parole generabili a partire dall'alfabeto A
 - A^* contiene anche la stringa vuota “” (ovvero una sequenza di zero simboli)

Alfabeti, parole, linguaggi

Un linguaggio L sull'alfabeto A è un qualsiasi sottoinsieme di A^* .

Parole italiane

“pwfnfkr”

“dog”

“mmto”

“siengs”

“**casa**”

“door”

“**porta**”

“foiasj”

Parole inglesi

“pwfnfkr”

“**dog**”

“mmto”

“siengs”

“casa”

“**door**”

“porta”

“foiasj”

Alfabeti, parole, linguaggi

Assumiamo $A=\{'0',\dots,'9'\}$

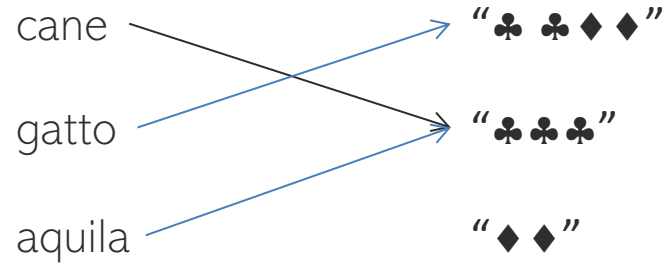
- Sia L l'insieme di parole che cominciano con '1':
 - "010", "123", "00237", "18923", "892003"
- Sia L l'insieme di parole in cui tutte le cifre '0',..., '9' di A occorrono lo stesso numero di volte
 - "02889", "", "22335", "4433", "0912837465", "09128374650918234765"
 - "4433" non è in L : la cifra '4' occorre 2 volte, la cifra '7' occorre 0 volte.
 - La parola vuota "" è in L : tutte le cifre occorrono 0 volte.
- Sia L l'insieme di parole per cui se due cifre occorrono almeno una volta allora occorrono lo stesso numero di volte.
 - "02889", "", "22335", "4433", "0912837465", "09128374650918234765"
- **Esercizio:** definire il linguaggio con cui usualmente denotiamo i numeri naturali

Codifica, decodifica

- Le parole di un linguaggio non sono altro che sequenze di simboli che non hanno alcun senso finché non forniamo un modo per interpretarle
 - “♣ ♣♦ ♠” ?
- Associare gli elementi di un insieme D alle parole di un linguaggio L viene detta codifica o rappresentazione di D.
- Ad esempio sia D l'insieme dei concetti cane, gatto e aquila (notate bene che sto parlando dei concetti non delle parole “cane”, “gatto” e “aquila”). Allora nella usuale codifica della lingua inglese
 - Cane → “dog”
 - Gatto → “cat”
 - Aquila → “eagle”
- Formalmente una codifica è una funzione totale $f: D \rightarrow L$
 - Se f non è suriettiva allora diremo che è ridondante
 - Se f non è iniettiva allora diremo che è ambigua

Codifica, decodifica

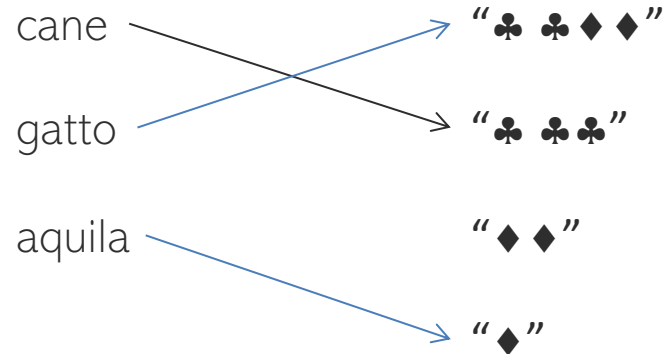
- Sia $D=\{\text{cane, gatto, aquila}\}$ e $L=\{\text{“♣ ♣ ♦ ♦”, “♣ ♣ ♣”, “♦ ♦”}\}$ e sia f rappresentata graficamente



- f è ambigua e ridondante

Codifica, decodifica

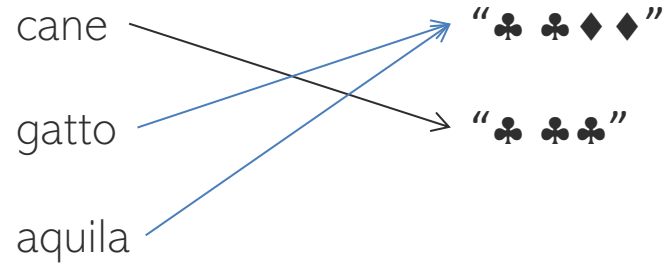
- Sia $D=\{\text{cane, gatto, aquila}\}$ e $L=\{\text{“♣ ♣ ♦ ♦”, “♣ ♣ ♣”, “♦ ♦”, “♦”}\}$ e sia f rappresentata graficamente



- f non è ambigua ma è ridondante

Codifica, decodifica

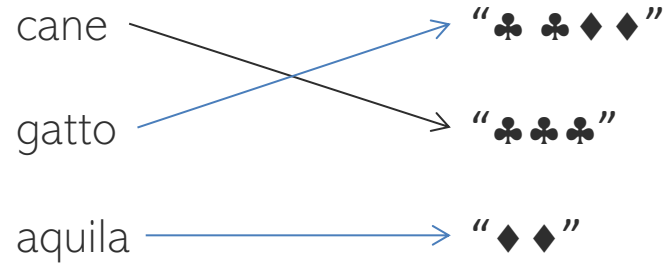
- Sia $D=\{\text{cane, gatto, aquila}\}$ e $L=\{“\clubsuit \clubsuit \spadesuit \spadesuit”, “\clubsuit \clubsuit \clubsuit”\}$ e sia f rappresentata graficamente



- f non è ridondante ma è ambigua

Codifica, decodifica

- Sia $D=\{\text{cane, gatto, aquila}\}$ e $L=\{“\clubsuit \clubsuit \diamond \diamond”, “\clubsuit \clubsuit \clubsuit”, “\diamond \diamond”\}$ e sia f rappresentata graficamente



- f non è ambigua e non è ridondante

Codifica, decodifica

- Sia N_D la cardinalità di D e N_L la cardinalità di L
 - Se $N_D > N_L$ qualsiasi codifica $f: D \rightarrow L$ è ambigua
 - Se $N_D < N_L$ qualsiasi codifica $f: D \rightarrow L$ è ridondante
- Inoltre è bene notare che
 - Non ambiguo significa che f è iniettiva (ad elementi distinti di D corrispondono elementi distinti di L)
 - Non ridondante significa che f è suriettiva (ogni stringa di L è la codifica di almeno un elemento di D)
 - Quindi se f è non ambigua e non ridondante allora f è iniettiva e suriettiva, quindi è una funzione biunivoca
- Una funzione $g: L \rightarrow D$ è invece detta una decodifica di D
- Se f è non ambigua (ovvero iniettiva) allora è invertibile. Quindi, induce naturalmente una decodifica $g=f^{-1}$

Proprietà delle codifiche

- Abbiamo già visto
 - ambigua/non ambigua
 - ridondante/non ridondante
- Altre proprietà
 - Economicità: numero di simboli utilizzati per unità di informazione
 - Semplicità nell'operazione di codifica e decodifica
 - Semplicità nell'eseguire operazioni sull'informazione codificata

Rappresentazione dei numeri naturali

- Occupiamoci ora delle possibili codifiche dei numeri naturali $0, 1, 2, \dots$
- Un codice ovvio è dato dal sistema di numerazione decimale basato su $A = \{ '0', \dots, '9' \}$.
- Tale sistema è un sistema posizionale ovvero ad ogni cifra di una parola è assegnato un peso differente a seconda della posizione nella sequenza. Ad esempio dato "4456"
 - '6' rappresenta sei unità (cifra meno significativa)
 - '5' rappresenta cinque decine
 - '4' rappresenta quattro centinaia
 - '4' rappresenta quattro migliaia (cifra più significativa)

Fin'ora ho estensivamente usato gli apici per distinguere le parole ("4456") da quello che rappresentano (il numero 4456). E' chiaro che usare gli apici ogni volta per rimarcare la distinzione fra numerali (parole) e numeri (concetti) può rendere molto pesante la trattazione. Quindi in seguito inizierò ad omettere gli apici qualora il contesto non generi ambiguità.

Rappresentazione in base 10

- Decomponendo in potenze di 10, il numerale 1024 rappresenta il numero

$$1 \times 10^3 + 0 \times 10^2 + 2 \times 10^1 + 4 \times 10^0$$

- Generalizzando, un numerale $c_{m-1}c_{m-2} \dots c_0$ rappresenta

$$\sum_{i=0}^{m-1} c_i \cdot 10^i$$

- Il sistema decimale è quindi una codifica posizionale su base 10. Tuttavia non è l'unica, ad esempio i babilonesi utilizzavano un sistema di numerazione su base 60 (sessagesimale)
- I dispositivi digitali per elaborare e memorizzare l'informazione possono trovarsi in due possibili stati che rappresentano la cifra '0' e la cifra '1'. Il sistema corrispondente è quello in base 2 (binario)

Rappresentazione in una base generica

- Ad ogni naturale $b > 1$ corrisponde una codifica in base b .
- L'alfabeto A_b consiste in b simboli distinti che corrispondono ai numeri $0, 1, \dots, b-1$.
- Analogamente al sistema decimale, un numerale $s_{m-1} \dots s_0$ di cifre di A_b rappresenta il numero

$$\sum_{i=0}^{m-1} s_i \cdot b^i$$

- Consideriamo ad esempio il sistema ottale (base 8). A_8 consiste nelle cifre '0', '1', ..., '7'

$$13 \longrightarrow 1 \cdot 8^1 + 3 \cdot 8^0 = 8 + 3 = 11$$

$$5201 \longrightarrow 5 \cdot 8^3 + 2 \cdot 8^2 + 0 \cdot 8^1 + 1 \cdot 8^0 = 5 \cdot 8^3 + 2 \cdot 8^2 + 1 \cdot 8^0 = 2560 + 128 + 1 = 2689$$

Rappresentazione in una base generica

A questo punto potrebbero sorgere delle ambiguità. Se qualcuno vi dicesse: *11 è un numero pari* pensereste che sia impazzito. Lui potrebbe ribattere: certo! infatti in base 5

$$11 \longrightarrow 1 \cdot 5^1 + 1 \cdot 5^0 = 5 + 1 = 6$$

E' chiaro che bisogna mettersi d'accordo su quale sistema di numerazione si adotta di volta in volta. Per questo useremo delle notazioni standard

- n denota un (generico) numero naturale, a prescindere dalla sua rappresentazione
- n_b denota il numerale che rappresenta il numero naturale n in base b
 - Esempio $1001_2 = 2^3 + 1 = 9$
- Nel caso della rappresentazione decimale per semplicità ometteremo il pedice. Ad esempio 236 denota il numero duecentotrentasei

Basi maggiori di 10

- Per basi $b < 10$ possiamo chiaramente ri-usare le usuali cifre '0', ..., '9'. Ad esempio, la codifica in base 6 utilizza le cifre '0', ..., '5' mentre quella in base 3 le cifre '0', '1' e '2' e così via.
- E per basi $b > 10$?
 - Si prendono in prestito le lettere dell'alfabeto
 - Per $b=16$ le cifre adottate sono '0', ..., '9', 'a', ..., 'f', dove:

$$a_{16} = 10 \quad b_{16} = 11$$

$$c_{16} = 12 \quad d_{16} = 13$$

$$e_{16} = 14 \quad f_{16} = 15$$

- Esempio:

$$b3c_{16} = 11 \cdot 16^2 + 3 \cdot 16^1 + 12 \cdot 16^0 = 2816 + 48 + 12 = 2876$$

Basi maggiori di 10

- E se le lettere dell'alfabeto non dovessero bastare?

𐤀 1	𐤁 11	𐤂 21	𐤃 31	𐤄 41	𐤅 51
𐤆 2	𐤇 12	𐤈 22	𐤉 32	𐤊 42	𐤋 52
𐤌 3	𐤍 13	𐤎 23	𐤏 33	𐤐 43	𐤑 53
𐤒 4	𐤓 14	𐤔 24	𐤕 34	𐤖 44	𐤗 54
𐤘 5	𐤙 15	𐤚 25	𐤛 35	𐤜 45	𐤝 55
𐤞 6	𐤟 16	𐤠 26	𐤡 36	𐤢 46	𐤣 56
𐤤 7	𐤥 17	𐤦 27	𐤧 37	𐤨 47	𐤩 57
𐤪 8	𐤫 18	𐤬 28	𐤭 38	𐤮 48	𐤯 58
𐤱 9	𐤲 19	𐤳 29	𐤴 39	𐤵 49	𐤶 59
𐤸 10	𐤹 20	𐤺 30	𐤻 40	𐤼 50	

Lunghezza di n rispetto ad una base

- Chiamiamo lunghezza di un numero naturale n rispetto ad una base b il numero di cifre che occorrono per rappresentare n in base b
 - la lunghezza di 101 rispetto a 2 è 7: $1100101_2 = 101$
 - la lunghezza di 101 rispetto a 10 è 3: $101_{10} = 101$
 - la lunghezza di 101 rispetto a 16 è 2: $65_{16} = 101$
- La lunghezza di un numerale decresce al crescere della base di codifica

Valore massimo rappresentabile

Fissato un certo numero di cifre, qual è il massimo numero rappresentabile in una data base?

Riferendoci alla base 10

- Con 1 cifra rappresentiamo i numeri da 0 a 9, ovvero 10^1-1
- Con 2 cifre i numeri da 0 a 99, ovvero 10^2-1
- Con 3 cifre i numeri da 0 a 999, ovvero 10^3-1
- Con m cifre i numero da 0 a 10^m-1

Quindi se indichiamo con $v_{\max}$ il massimo numero rappresentabile con m cifre in base 10 abbiamo

$$v_{\max} = 10^m - 1$$

Valore massimo rappresentabile

- E per una base diversa da 10, quale è il massimo valore rappresentabile con m cifre?
- Consideriamo, ad esempio, il caso $b=2$ e $m=4$, il massimo valore rappresentabile corrisponde al numerale 1111

$$1111_2 = 2^3 + 2^2 + 2^1 + 2^0 = 15 = 2^4 - 1$$

- Per una generico b e m il massimo numero rappresentabile si ottiene concatenando m volte la cifra “più alta” (che corrisponde al valore $b-1$). Quindi:

$$\begin{aligned} v_{max} &= (b-1)b^{m-1} + (b-1)b^{m-2} + \dots + (b-1)b^1 + (b-1)b^0 = \\ &= (b \cdot b^{m-1} - b^{m-1}) + (b \cdot b^{m-2} - b^{m-2}) + \dots + (b \cdot b^1 - b^1) + (b \cdot b^0 - b^0) = \\ &= b^m - b^{m-1} + b^{m-1} - b^{m-2} + \dots + b^2 - b + b - 1 = b^m - 1 \end{aligned}$$

- Esempio il massimo numero rappresentabile con 4 cifre in base 3 è $3^4 - 1 = 80$

Cifre necessarie per rappresentare n

- Nella slide precedente avevamo il numero di cifre m e volevamo sapere quale è il *massimo numero rappresentabile* in una certa base b .
- Consideriamo il problema inverso: abbiamo un valore n e ci chiediamo quante *cifre m occorrono per rappresentarlo*.
- Chiaramente il massimo numero rappresentabile con m cifre dovrà essere maggiore o uguale a n

$$b^m - 1 \geq n$$

$$b^m \geq n + 1$$

$$m \geq \log_b(n + 1)$$

- In particolare cerchiamo il più piccolo intero m tale che $m \geq \log_b(n + 1)$

$$m = \lceil \log_b(n + 1) \rceil$$

Cifre necessarie per rappresentare n

- Notazione: dato un reale x
 - $\lceil x \rceil$ denota l'approssimazione intera per eccesso
 - $\lfloor x \rfloor$ denota l'approssimazione intera per difetto
- Esempi:
 - $\lceil 14.78 \rceil = 15$
 - $\lfloor 9.88 \rfloor = 9$
- In qualche testo potrete trovare $m = \lfloor \log_b n \rfloor + 1$
 - Qualcuno si sbagliato?
- **Esercizio:** dimostrare che per ogni n e b

$$\lceil \log_b(n + 1) \rceil = \lfloor \log_b n \rfloor + 1$$

- **Esercizio:** dimostrare che per ogni n e b $\lceil \log_b(n+1) \rceil = \lfloor \log_b n \rfloor + 1$
- Prima osservazione:



Se $x < y$ ALLORA $\lfloor x \rfloor + 1 \leq \lceil y \rceil$

Quindi $\lfloor \log_b n \rfloor + 1 \leq \lceil \log_b(n+1) \rceil$

- Supponiamo per assurdo che $\underbrace{\lfloor \log_b n \rfloor + 1}_{\text{chiamiamolo } c} < \lceil \log_b(n+1) \rceil$

- Essendo c un intero allora c deve essere compreso fra



- Ma allora avremmo:

$$\log_b n < c < \log_b (n+1)$$



$$b^{\log_b n} < b^c < b^{\log_b (n+1)}$$

$$n < b^c < n+1$$

$$\begin{array}{l} b \in \mathbb{N} \\ c \in \mathbb{N} \end{array} \Rightarrow b^c \in \mathbb{N}$$

b^x

- Quindi esisterebbe un numero naturale fra n e $n+1$, **assurdo!**

Relazione fra due basi diverse

- Consideriamo due sistemi di numerazione nelle basi a e b . Dato un naturale n , quale relazione intercorre fra m_a e m_b ?
- Per ogni base c , $\lfloor \log_c n \rfloor + 1 = \log_c n + 1$
- Quindi considereremo le approssimazioni

$$\begin{aligned}m_a &\simeq \log_a n \\m_b &\simeq \log_b n\end{aligned}$$

- Ricordando che

$$\log_a n = \frac{\log_b n}{\log_b a}$$

- il rapporto fra le lunghezze m_b e m_a è costante (indipendente da n) e pari a $\log_b a$

$$m_a \simeq \log_a n = \frac{\log_b n}{\log_b a} = \frac{m_b}{\log_b a}$$

Digressione: base unaria

- Ricordate che quando abbiamo introdotto i sistemi di numerazione abbiamo assunto la base $b \geq 2$.
- Perché non è possibile considerare una numerazione unaria? Certo! Ma ci sono delle controindicazioni...
 - La base unaria consta di una solo cifra 1 (detta in gergo matematico *mazzarella*)
 - Una mazzarella rappresenta 0, due mazzarelle 1, ..., $n+1$ mazzarelle rappresentano n
 - $IIIII_1 = 5$
 - Notate che a prescindere dalla posizione ogni mazzarella vale 1, quindi questo sistema non è posizionale
 - Non potrebbe essere altrimenti visto che 1 elevato ad una qualsiasi potenza è sempre uguale a 1!

Digressione: base unaria

Andiamo a confrontare le lunghezze dei numerali con una base $b \geq 2$

$$m_1 = n + 1 = b^{\log_b n} + 1 \simeq b^{m_b} + 1 \simeq b^{m_b}$$

Questo vuol dire che la codifica di n in base unaria cresce esponenzialmente rispetto alla codifica in base b !

La base unaria non è un buon sistema di rappresentazione

Codifica ottimale

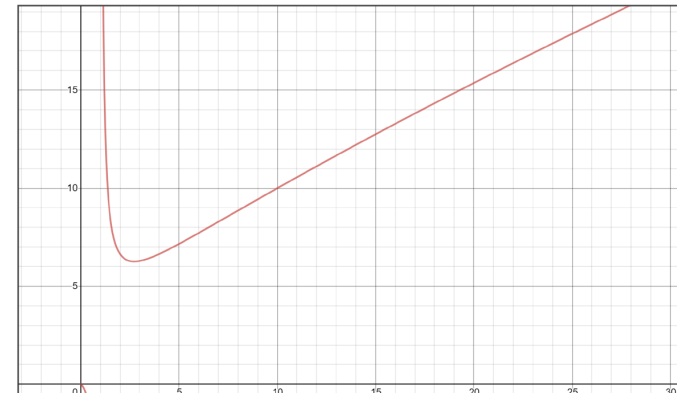
- Ritorniamo alle basi ammissibili ($b \geq 2$)
- Abbiamo visto che più grande è la base b , minore è il numero di cifre che occorrono per rappresentare un numero
- Quindi il codice binario è il sistema di codifica meno economico fra quelli ammissibili
 - Per esempio $\log_2 10 \cong 3.32$ questo vuol dire che per rappresentare un numero n in binario occorrono *circa il triplo* delle cifre che occorrono per rappresentare lo stesso n nel sistema decimale
 - *Perché i computer adottano la codifica binaria?*

Codifica ottimale

- Non bisogna tener presente solo la lunghezza di codifica ma anche il fatto che un calcolatore per operare in una certa base b deve poter rappresentare tutte le cifre di quella base (da 0 a $b-1$), *quindi gli servono b differenti stati*.
- Una nozione di costo di una codifica che tiene conto anche di questo fattore è data dal prodotto

$$b \cdot m_b \simeq b \cdot \log_b n$$

- Si può mostrare che il valore di b che minimizza tale costo è il numero di Nepero e $\cong 2.7$, quindi le codifiche che si avvicinano di più a tale valore ottimale sono quelle in base 2 e 3



Cambiamento di base

- Problema: dato n rappresentato in base a , quale è la sua rappresentazione in base b ?
- Supponiamo di saper fare le quattro operazioni in base a
- L'algoritmo di cambiamento di base consiste nel dividere ripetutamente n (espresso in base a) per b finché il quoziente non risulti uguale a zero. La sequenza di resti ottenuti (compresi tra 0 e $b-1$) è la codifica dalla cifra meno significativa a quella più significativa di n in base b
- Esempio: codificare 251_{10} in base 3

$$251/3 = 83 \quad \text{resto: } 2$$

$$83/3 = 27 \quad \text{resto: } 2$$

$$27/3 = 9 \quad \text{resto: } 0$$

$$9/3 = 3 \quad \text{resto: } 0$$

$$3/3 = 1 \quad \text{resto: } 0$$

$$1/3 = 0 \quad \text{resto: } 1$$

$$251_{10} = 100022_3$$

Cambiamento di base

- Esempio: codificare 333_7 in base 9
- Problema: non siamo molto allenati con la divisione in base 7. Meglio affrontare il problema in due passi:

- Codifico 333_7 in base 10

$$333_7 = 3 \cdot 7^2 + 3 \cdot 7^1 + 3 \cdot 7^0 = 171$$

- Codifico 171_{10} in base 9

$$171/9 = 19 \quad \text{resto: } 0$$

$$19/9 = 2 \quad \text{resto: } 1$$

$$2/9 = 0 \quad \text{resto: } 2$$

$$333_7 = 210_9$$

Codifica binaria e esadecimale

- Abbiamo detto che i componenti digitali operano in codice binario.
- Tuttavia la rappresentazione in binario non è molto human-friendly perché genera codici piuttosto lunghi
 - $599 = 1001010111_2$
- Si potrebbe pensare di usare la nostra base naturale (10). Questo comporta usare il precedente algoritmo per codifica/decodifica
 - Non proprio agevole
 - Non proprio velocissimo
 - Il problema è che 10 non è una potenza di 2

Codifica binaria e esadecimale

- La codifica/decodifica in una base m potenza di 2 permette invece di usare qualche truccetto che facilita la codifica e decodifica.
- le cifre utilizzate nel sistema esadecimale sono '0', ..., '9', 'a', ..., 'f'.
- Inoltre, poiché $16 = 2^4$ è possibile codificare ogni cifra del sistema esadecimale mediante 4 bit
- Viceversa 4 bit del sistema binario corrispondono ad una cifra esadecimale

Codifica binaria e esadecimale

- Codifica delle cifre esadecimali in binario

0 $\longrightarrow$ 0000	8 $\longrightarrow$ 1000
1 $\longrightarrow$ 0001	9 $\longrightarrow$ 1001
2 $\longrightarrow$ 0010	<i>a</i> $\longrightarrow$ 1010
3 $\longrightarrow$ 0011	<i>b</i> $\longrightarrow$ 1011
4 $\longrightarrow$ 0100	<i>c</i> $\longrightarrow$ 1100
5 $\longrightarrow$ 0101	<i>d</i> $\longrightarrow$ 1101
6 $\longrightarrow$ 0110	<i>e</i> $\longrightarrow$ 1110
7 $\longrightarrow$ 0111	<i>f</i> $\longrightarrow$ 1111

- Notate che questa codifica corrisponde alla codifica dei rispettivi numeri con possibili 0 non significativi

$$0111_2 = 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 7$$

Da esadecimale a binario

- Dato un numerale esadecimale per codificarlo in binario è sufficiente giustapporre le codifiche delle singole cifre (eliminando eventualmente zeri non significativi)
- Esempi:
 - $4c9f_{16} \rightarrow 0100\ 1100\ 1001\ 1111 \rightarrow 100110010011111_2$
 - $b2a_{16} \rightarrow 1011\ 0010\ 1010 \rightarrow 101100101010_2$
 - $157_{16} \rightarrow 0001\ 0101\ 0111 \rightarrow 101010111_2$
- Nota bene che $157_{16} = 1 \cdot 16^2 + 5 \cdot 16^1 + 7 \cdot 16^0 = 343$

Da binario a esadecimale

- Per il passaggio di base da binario a esadecimale si effettua la codifica inversa avendo cura di aggiungere eventuali zeri non significativi in modo che la lunghezza del numerale in binario sia multipla di 4
- Esempi:
 - $10_2 \rightarrow 0010 \rightarrow 2_{16}$
 - $10011_2 \rightarrow 0001\ 0011 \rightarrow 13_{16}$
 - $111110010101100_2 \rightarrow 1111\ 1001\ 0101\ 1100 \rightarrow f95c_{16}$
- Nota: comunemente un numerale esadecimale viene indicato con il prefisso 0x
 - $f95c_{16} \rightarrow 0xf95c$

Rappresentazione registri

- Un computer le grandezze numeriche sono elaborate mediante sequenze di simboli di lunghezza fissa dette parole
- Poichè una cifra esadecimale codifica 4 bit:
 - 1 byte (8 bit) → 2 cifre esadecimali
 - 4 byte (32 bit) → 8 cifre esadecimali
 - 8 byte (64 bit) → 16 cifre esadecimali

Esercizi

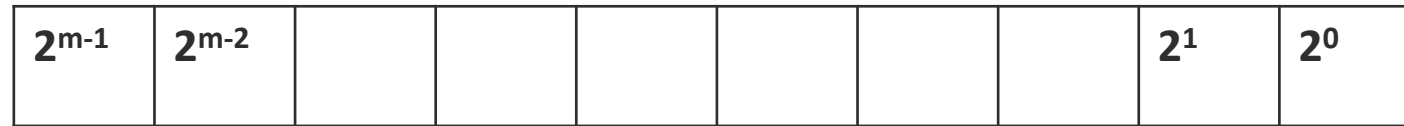
- Convertire da base 2 a base 10:
 - 0110011
 - 10101100
 - 1100110011
- Convertire in base 10 i seguenti numeri:
 - 102210_3
 - 431204_5
 - 5036_7
 - $198A1_{12}$

Esercizi

- Convertire da base 10 alla base indicata i seguenti numeri:
 - 7562 base 8
 - 1938 base 16
 - 205 base 16
 - 175 base 2
- In un registro a 32 bit è memorizzato il valore 0xF3A7C2A4. Esprimere il contenuto del registro in base 2
- Convertire da base 2 a base 16:
 - 10010
 - 11010101
 - 10010011
- Scrivere in babilonese il numero 4000

Numeri con parole di lunghezza fissa

- I registri dei moderni calcolatori sono tipicamente parole di 32 o 64 bit
- Con una parola di lunghezza fissa sono rappresentabili un numero finito di naturali.



- Nel caso di parole a 64 bit sono rappresentabili i numeri da 0 a $2^{64}-1$
- Chiaramente, poiché ogni numero deve essere rappresentato dallo stesso numero di cifre, occorre ricorrere necessariamente a zeri non significativi

Numeri con parole di lunghezza fissa

- Supponiamo di avere una macchina che opera con parole di 16 bit (come l'Atari St di quando ero piccolo)
 - il numero 9 sarà quindi rappresentato come: 0000 0000 0000 1001
- Operazioni come l'addizione o la moltiplicazione possono produrre numeri troppo grandi per essere rappresentati con un lunghezza fissata. In questo caso parleremo di trabocco o *overflow*
- Torniamo alla nostra macchina a 16 bit e supponiamo di voler elevare 1024 al quadrato. Questo produrrà un trabocco, infatti:

$$1024^2 = 1024 \cdot 1024 = 2^{10} \cdot 2^{10} = 2^{20} > 2^{16} - 1$$

Somma in binario

L'operazione di somma in binario è algebricamente analoga a quella decimale con la differenza che il riporto si ha quando si eccede 1 (invece che 9)

$$\begin{array}{r} \text{11} \\ 4277 \\ + 5499 \\ \hline 9776 \end{array} \quad \leftarrow \text{carries} \rightarrow \quad \begin{array}{r} \text{11} \\ 1011 \\ + 0011 \\ \hline 1110 \end{array}$$

(a) (b)

Overflow: al termine della somma ho un riporto di 1

$$\begin{array}{r} \text{11} \quad \text{1} \\ 1101 \\ + 0101 \\ \hline 1\boxed{0010} \end{array}$$

Moltiplicazione in binario

Anche la moltiplicazione in binario è analoga a quella decimale

$$\begin{array}{r} 0101 \times \\ 0011 = \\ \hline 0101 \\ 0101 = \\ 0000 == \\ 0000 === \\ \hline 0001111 \end{array}$$

Rappresentazione di interi

- Occupiamoci ora della rappresentazione di numeri interi $\dots, -2, -1, 0, 1, 2, \dots$ mediante parole di lunghezza fissata
- Chiaramente dobbiamo codificarne non solo il valore assoluto ma anche il segno
- Le principali rappresentazioni di numeri interi sono:
 - Rappresentazione con segno
 - Complemento all'intervallo (complemento a due)

Rappresentazione con segno

- Si supponga che le parole siano di lunghezza m e la base sia b
- In tale rappresentazione la cifra più significativa di una parola rappresenta il segno. Per convenzione 0 rappresenta il segno più, 1 rappresenta il segno meno
- Le rimanenti $m-1$ cifre sono usate per rappresentare il valore assoluto di un intero

+/-	2^{m-2}							2^1	2^0
------------	-----------------------------	--	--	--	--	--	--	-------------------------	-------------------------

- Ad esempio si considerino parole binarie di lunghezza 4
 - $0100 \rightarrow +100 \rightarrow 4$
 - $1011 \rightarrow -011 \rightarrow -3$
 - Il più grande numero rappresentabile è 0111 (ovvero 7)
 - Il più piccolo numero rappresentabile è 1111 (ovvero -7)
 - Lo zero ha due possibili rappresentazioni: 0000 e 1000

Rappresentazione con segno

- Poiché $m-1$ cifre sono utilizzate per rappresentare il valore assoluto, il range di numeri codificabili è
$$-(b^{m-1} - 1), \dots, 0, \dots, b^{m-1} - 1$$
- Svantaggio: nelle operazioni di addizione o sottrazione occorre controllare il segno e i valori assoluti dei due operandi per determinare il segno del risultato.
 - $0010 + 0001$
 - sono entrambi positivi quindi il segno sarà positivo $\rightarrow 0011$
 - $0101 + 1010$
 - il primo è positivo mentre il secondo è negativo, il valore assoluto del primo è maggiore del valore assoluto del secondo quindi il segno sarà positivo $\rightarrow 0011$
 - $1100 + 0011$
 - il primo è negativo mentre il secondo è positivo, il valore assoluto del primo è maggiore di quello del secondo, quindi il segno sarà negativo $\rightarrow 1001$
 - $1011 - 1010$
 - Entrambi sono negativi ma il valore assoluto del secondo è minore di quello del primo. Quindi occorre sottrarre 010 da 011 cambiando il bit del segno $\rightarrow 1001$

Complemento a 2

- La rappresentazione è analoga a quella unsigned con la differenza che il bit più significativo corrisponde a -2^{m-1} invece che 2^{m-1}

-2^{m-1}	2^{m-2}							2^1	2^0
------------	-----------	--	--	--	--	--	--	-------	-------

- Conosideriamo ad esempio la parola di 8 bit
In complemento a due rappresenta il numero:

1	0	0	1	1	1	0	0
---	---	---	---	---	---	---	---

-128	+0	+0	+16	+8	+4	+0	+0
------	----	----	-----	----	----	----	----

 = -100

Complemento a 2

- Lo zero ha una unica rappresentazione

0	0							0	0
---	---	--	--	--	--	--	--	---	---
- Minimo valore rappresentabile: -2^{m-1} →

1	0	0					0	0	0
---	---	---	--	--	--	--	---	---	---
- Massimo valore rappresentabile: $2^{m-1} - 1$ →

0	1	1					1	1	1
---	---	---	--	--	--	--	---	---	---
- Il range di rappresentazione è $[-2^{m-1}, 2^{m-1} - 1]$
- Essendo -2^{m-1} in valore assoluto il «peso» più grande, se un numero inizia con 1 allora è negativo altrimenti è positivo

Complemento a 2

- La somma avviene come per i numeri unsigned. Consideriamo ad esempio parole di 4 bit
 - $-2+1 = 1110 + 0001 = 1111 = -1$
 - $-7+7 = 1001 + 0111 = 0000$ (con riporto 1) = 0
- Per complementare un numero occorre invertire ogni cifra e sommare 1
 - Rappresentare il numero -2 e -7 con 4 bit
 - $2=0010 \rightarrow -2= 1101 + 0001 = 1110$
 - $7=0111 \rightarrow -7= 1000 + 0001 = 1001$
 - Attenzione! questo non vale per -2^{m-1} il cui complemento non è rappresentabile con m bit (i numeri positivi arrivano a $2^{m-1} - 1$):
 - $-2^3 = 1000 \rightarrow 0111 + 0001 = 1000$

Overflow nel complemento a 2

- Sommare un numero negativo e uno positivo non genera overflow
- L'esempio $-7+7$ mostra come l'overflow *non avviene come per gli unsigned* quando ho riporto finale di 1
- L'overflow avviene quando entrambi gli operandi sono negativi (primo bit=1) o entrambi positivi (primo bit=0) è il risultato ha segno opposto
 - $4+5=0100+0101 = 1001 = -7$
- Per estendere un numero in una rappresentazione con più bit basta riprodurre a sinistra il bit più significativo
 - $5=0101 \rightarrow 00000101$
 - $-4=1100 \rightarrow 11111100$

Esercizi

- Fornire la rappresentazione binaria in complemento a 2 ad 8 bit del numero -15
- Fornire la rappresentazione binaria in complemento a 2 ad 8 bit del numero -109
- Eseguire la somma dei numeri 5 e -32 espressa in complemento a 2 ad 8 bit
- Convertire in esadecimale il numero -53248 espresso in complemento a 2 ad 16 bit

Binary Coded Decimal

- L'elettronica degli elaboratori è binaria mentre la mente umana è abituata a ragionare in decimale.
- I codici Binary Coded Decimal hanno lo scopo di fornire una naturale rappresentazione binaria del sistema numerico decimale.
- Essendo 10 i simboli da codificare ('0', ..., '9') avremo bisogno di 4 bit.
- Notate che con 4 bit consentono di avere $2^4=16$ combinazioni che in binario puro corrispondono ai numeri 0,1,2,...,9,10,...,15
- Poiché le combinazioni da 10 a 15 non si usano, la codifica BCD è *ridondante*

Binary Coded Decimal

Cifra decimale	Cifra BCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

i codici 1010, 1011, 1100, 1101, 1110, 1111 non sono utilizzati (codifica ridondante)

Numeri decimali a più cifre si codificano giustapponendo le codifiche cifra per cifra

Esempio:

$$23 = 0010\ 0011_{\text{BCD}}$$

Nota che

$$23 = 00100011_{\text{BCD}} \neq 00100011_2 = 35$$

Anche le somme differiscono:

$$1000_{\text{BCD}} + 0011_{\text{BCD}} = 8 + 3 = 11 = 00010001_{\text{BCD}}$$

e non 1011 (che è un codice non utilizzato)

Binary Coded Decimal

- Naturalmente, la codifica BCD viene usata solo in funzione di interfaccia per rendere comprensibile ad operatori umani i risultati di una elaborazione numerica binaria. La codifica BCD del numero è, infatti, una operazione propedeutica alla sua visualizzazione su un display numerico decimale (es. display a sette segmenti). I quattro bit di ciascuna cifra codificata BCD vengono inviati ad un circuito di decodifica che provvederà ad attivare i segmenti della cifra corrispondente.
- E' opportuno sottolineare che, mentre la codifica binaria è il primo passo per approdare ad un sistema numerico che porta poi all'aritmetica binaria, non esiste una aritmetica BCD.

Numeri con virgola

- Vediamo come rappresentare (approssimazioni di numeri) reali.
- Consideriamo un numero con virgola nella base naturale 10

$$c_{m-1}c_{m-2}\cdots c_0, c_{-1}\cdots c_{-k} = c_{m-1} \cdot 10^{m-1} + c_{m-2} \cdot 10^{m-2} + \cdots c_0 \cdot 10^0 + c_{-1} \cdot 10^{-1} + \cdots + c_{-k} \cdot 10^{-k}$$

- Per una generica base b abbiamo la generalizzazione

$$\sum_{i=-k}^{h-1} c_i \cdot b^i$$

Cambiamenti di base con virgola

- Per il cambiamento di un numero x da una base a ad una b si procede separatamente per la parte intera e per quella frazionaria
- Per la parte intera l'algoritmo chiaramente è quello visto in precedenza
- Per la parte frazionaria in procedimento è l'inverso:
 - si moltiplica la parte frazionaria di x per b (entrambi codificati in base a). La parte intera del risultato sarà un numero da 0 a $b-1$ che, convertito in base b , costituisce la prima cifra frazionaria di x in base b .
 - La parte frazionaria del risultato f si moltiplica ancora per b e la nuova parte intera, convertita in base b , costituisce la seconda cifra frazionaria.
 - Si procede iterativamente finché la parte frazionaria del risultato è zero o si è raggiunta la precisione desiderata

Cambiamenti di base con virgola

convertire in binario 0,625

$$0,625 \cdot 2 = 1,250 \quad i = 1 \quad f = 0,250 \quad c_{-1} = 1$$

$$0,250 \cdot 2 = 0,500 \quad i = 0 \quad f = 0,500 \quad c_{-2} = 0$$

$$0,500 \cdot 2 = 1,000 \quad i = 1 \quad f = 0,000 \quad c_{-3} = 1$$

$$0,625 = 0,101_2$$

Convertire $0,65_8$ in base 7 fino alla 3 cifra significativa

- Convertiamo prima in decimale
 - $6 \times 8^{-1} + 5 \times 8^{-2} = 6 \times 0,125 + 5 \times 0,015625 = 0,75 + 0,078125 = 0,828125$
- Convertiamo $0,828125$ in base 7: $0,554_7$
 - $0,828125 \times 7 = 5,796875 \quad 5$
 - $0,796875 \times 7 = 5,578125 \quad 5$
 - $0,578125 \times 7 = 4,046875 \quad 4$

Rappresentazione in virgola fissa

- Ci occupiamo ora di rappresentare numeri positivi frazionari con parole (binarie) di lunghezza fissata m
- Nella rappresentazione in virgola fissa si suddividono gli m bit in due sottoparole
 - i primi h bit (con $h \leq m$) sono dedicati alla codifica della parte intera
 - i rimanenti $k = m - h$ bit rappresentano la parte frazionaria
- Supponiamo di far uso di parole a 32 bit e di dedicare 20 bit per la parte intera e 12 per la parte frazionaria:
 - Massimo intero codificabile: $2^{20} - 1$
 - Con 12 bit per la parte frazionaria si codificano circa 3 cifre decimali (ricordate $\log_2 10 = 3,32$)

Esercizi

- Fornire la rappresentazione binaria in virgola fissa del numero 7.25 con 4 bit per la parte intera e 4 bit di parte frazionaria
- Riportare in codice esadecimale la rappresentazione binaria in virgola fissa del numero 2.33 con 4 bit per la parte intera e 4 bit di parte frazionaria, trascurando l'eventuale resto
- Fornire la rappresentazione binaria in virgola fissa del numero 55.4121 con 8 bit per la parte intera e 4 bit di parte frazionaria, trascurando l'eventuale resto
- Eseguire la somma $12.25 + 5.5$ nella rappresentazione binaria in virgola fissa con 5 bit per la parte intera e 3 per quella frazionaria
- Eseguire la somma $9.875 + 10.5$ nella rappresentazione binaria in virgola fissa con 5 bit per la parte intera e 3 per quella frazionaria

Rappresentazione in virgola mobile

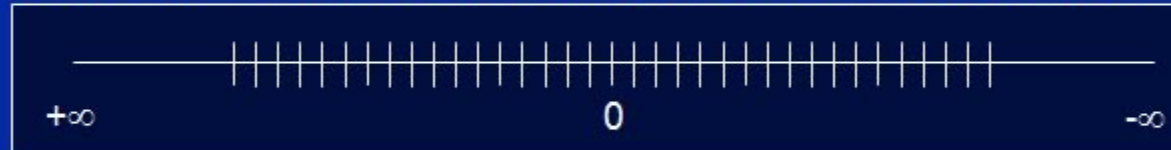
- Nella rappresentazione in virgola fissa disponendo di parole a 64 bit e supponendo di dedicarle tutte per la rappresentazione della parte frazionaria riusciremmo a codificare circa 18 cifre decimali
- Alcune applicazioni scientifiche operano con valori ancora più piccoli, altre invece con valori molto grandi
- In generale, fissare a priori la lunghezza della parte intera h e di quella frazionaria k costituisce una scelta rigida
- Per ovviare a queste difficoltà è stata introdotta una rappresentazione detta in virgola mobile

Rappresentazione in virgola mobile

Floating-Point: non-uniform distribution (variable precision)



IQ Fractions: uniform distribution (same precision everywhere)



- ◆ Both floating-point and IQ formats have 2^{32} possible values on the number line
- ◆ It's how each distributes these values that differs

Rappresentazione in virgola mobile

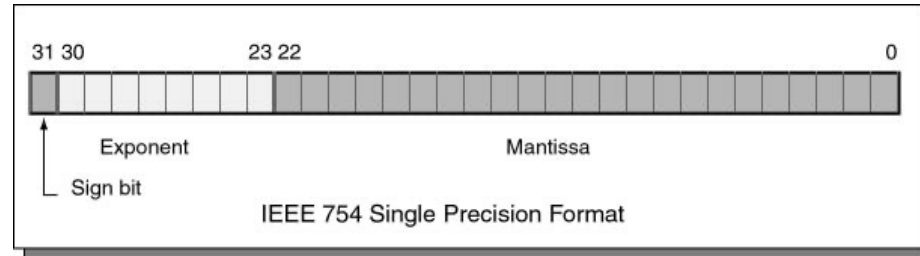
- Questa sfrutta la rappresentazione di un numero in una data base b :

$$x = (-1)^s m b^e$$

- s determina il segno: 0 positivo, 1 negativo
 - m è detta matissa
 - e è detto esponente
- Un numero reale è quindi rappresentato da una triple (s,m,e) . Notate però che vi sarebbero infiniti modi di rappresentare x . Ad esempio, per 34,67 in base 10:
 - 3467×10^{-2}
 - $3,467 \times 10$
 - $0,3467 \times 10^2$
- Useremo la rappresentazione scientifica in cui $b > m \geq 1$
- In binario questo vuol dire che la mantissa è sempre del tipo $1,xyz$
 - Chiaramente nella rappresentazione della mantissa non sprecherò memoria per rappresentare il primo bit "1" e lo considero sottointeso

Standard IEEE 754

- Una rappresentazione largamente adottata è quella dell'Institute of Electrical and Electronical Engineering (IEEE)
- Prevede 4 diversi formati per calcoli in singola o doppia precisione di tipo semplice o esteso (raramente usato)
- Descriveremo i tipi semplici
 - Singola precisione: 32 bit totali, 1 per il segno, 23 per la mantissa e 8 per l'esponente



- Doppia precisione: 64 bit totali, 1 per il segno, 52 per la mantissa e 11 per l'esponente

Dettagli dello Standard IEEE

- Riguardo all'esponente, il complemento a 2 inverte l'ordine fra positivi e negativi (i numerali associati ai negativi sono *maggiori* di quelli associati ai positivi)
- Per ovviare a questo difetto si usa nello standard IEEE la “rappresentazione polarizzata”
- Dati k bit assegnati all'esponente, il più grande esponente rappresentabile (in complemento a 2) è $P = 2^{k-1} - 1$
- In rappresentazione polarizzata, un valore e (compreso tra 0 e $2^k - 1$) codifica l'esponente $e' = e - P$
- Ad esempio sia $k=3$ ($P = 2^2 - 1 = 3$):

$e = 0$ [000] $\rightarrow e' = -3$	$e = 1$ [001] $\rightarrow e' = -2$	$e = 2$ [010] $\rightarrow e' = -1$	$e = 3$ [011] $\rightarrow e' = 0$
$e = 4$ [100] $\rightarrow e' = 1$	$e = 5$ [101] $\rightarrow e' = 2$	$e = 6$ [110] $\rightarrow e' = 3$	$e = 7$ [111] $\rightarrow e' = 4$

- Quindi ora gli esponenti sono codificati in ordine crescente da -3 (000_2) a 4 (111_2)

Dettagli dello Standard IEEE

- Per la mantissa si adotta la rappresentazione scientifica $1,xyz\dots$
- Riassumendo ad (s, m, e) è associato il numero

$$x = (-1)^s \cdot 1, m \cdot 2^{e-P}$$

- IEEE 754 precisione singola: 8 bit per l'esponente
 - $P = 2^7 - 1 = 127$
- IEEE 754 precisione doppia: 11 bit per l'esponente
 - $P = 2^{10} - 1 = 1023$
- Problema con questa rappresentazione non riesco a rappresentare lo 0. Per questo lo standard IEEE considera delle eccezioni.

Dettagli dello Standard IEEE

precisione singola

	e=0 (00000000 ₂)	e=1,...,254	e=255 (11111111 ₂)
m=0	$(-1)^s \times 0$	$(-1)^s \times 1,0 \times 2^{e-127}$	$(-1)^s \infty$
m≠0	$(-1)^s \times 0,m \times 2^{-126}$	$(-1)^s \times 1,m \times 2^{e-127}$	NaN (indefinito)

Per $m = 0$ ed $e = 0$, si hanno due rappresentazioni dello 0, a seconda che s sia 1 (segno negativo) o 0 (segno positivo).

Le combinazioni che corrispondono ad $m = 0$ ed $e = 255$ sono la rappresentazione di $\pm \infty$.

*Per $m \neq 0$ ed $e = 0$ è prevista una rappresentazione per numeri molto vicini allo 0. Ricordate che m non è vincolato a iniziare con 1. Quindi se usassimo la rappresentazione usuale avrei, assumendo $s=0$, otterrei numeri maggiori di 2^{-127} : $1,m \times 2^{-127} > 2^{-127}$
Invece per $m=00...01$: $0,0...01 \times 2^{-126} = 2^{-23} \times 2^{-126} = 2^{-149}$*

Esempi standard IEEE

- Il numero decimale 1021 è rappresentato dalla tripla (s, m, e):
(0; 111 1111 0100 0000 0000 0000; 1000 1000)

s= 0 perché il numero è positivo.

La rappresentazione binaria di 1021 è:

$$1\ 111\ 1111\ 01 = 1,111\ 1111\ 01 \times 2^9$$

$$m = 111\ 1111\ 0100\ 0000\ 0000\ 0000$$

Avendo 8 bit a disposizione per l'esponente, $P = 2^{8-1} - 1 = 2^7 - 1 = 127$.

L'esponente e si ricava invertendo la relazione di definizione della costante di polarizzazione:

$$e = e' + P = 9 + 127 = 136$$

che, convertito in binario, da 1000 1000

Esercizi

- Rappresentare nel formato IEEE 754
 - 1.25
 - 10
 - -0.625
 - 1007
 - 3.875

potrete verificare la correttezza degli esercizi svolti tramite
<https://www.h-schmidt.net/FloatConverter/IEEE754.html>

- -0.625

- Convertire in binario

$$0,625 \times 2 = 1,25 \quad 1$$

$$0,25 \times 2 = 0,5 \quad 0$$

$$0,5 \times 2 = 1,0 \quad 1$$

$$0.625 = 0,101_2$$

- Normalizzare nel formato scientifico 1,xyz...

$$0,101 = 1,01 \times 2^{-1}$$

- Quindi ottengo la rappresentazione binaria in virgola mobile

$$(-1)^1 \ 1,01 \ 2^{-1}$$

- Conversione nello standard IEEE 754

- $s=1$

- $m= 010 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000$

- $e = 127 - 1 = 126 = 0111 \ 1110$

Operazioni in virgola mobile

- La moltiplicazione fra due numeri n_1 ed n_2 rappresentati in virgola mobile dalle triple (s_1, m_1, e_1) ed (s_2, m_2, e_2) ha per risultato il numero rappresentato dalla tripla: (s, e, m) in cui:
 - $s = 0$ se $s_1 = s_2$ oppure: $s = 1$ se $s_1 \neq s_2$
 - $e = e_1 + e_2$
 - $m = m_1 \times m_2$
 - Dopo l'operazione di solito è necessaria la normalizzazione del risultato
- La divisione si effettua con regole analoghe.
- L'addizione e la sottrazione sono più complesse perché prima di effettuarle bisogna rendere uguali gli esponenti.
 - Durante questa operazione se i numeri sono uno molto grande ed uno molto piccolo, per effetto dello scorrimento delle mantisse per pareggiare gli esponenti, si possono perdere cifre significative.

Perdita di cifre significative

- Vediamo con un esempio perché si possono perdere cifre significative effettuando una somma. Per semplicità, considereremo una coppia di numeri decimali espressi attraverso una mantissa di 4 cifre ed un esponente di una sola cifra:

$$n1 = 1,3435 \times 10^3 \text{ ed } n2 = 1,9970 \times 10^5.$$

- Per effettuare la somma si può portare l'esponente di $n1$ da 3 a 5. Siccome ciò equivale a moltiplicare di fattore 100, per mantenere il valore costante occorre contemporaneamente dividere per 100 la mantissa:

$$1,3435 \times 10^3 = 0,013435 \times 10^5$$

- Poiché le cifre della mantissa sono 4, occorre effettuare un arrotondamento. Per difetto otterremmo 0,0134.
- La somma dei due numeri risulta: $2,0104 \times 10^5$ (invece di $2,010435 \times 10^5$)

Codifica caratteri alfa-numerici

- I calcolatori, nonostante il nome italiano (in francese si chiamano *ordinateur*) sono spesso utilizzati per manipolare informazioni non numeriche.
- Si parla di caratteri "alfanumerici" per sottolineare che in un testo sono presenti:
 - caratteri alfabetici (a,b,c,d,...)
 - caratteri numerici (0,...,9)
 - segni di punteggiatura (!,?,...)
 - simboli particolari vario tipo (£, &, @, ...)
- I processori moderni non hanno istruzioni specifiche per testi. Quindi, si usano codifiche da testo a numeri interi
- Un testo è una sequenza di caratteri. I codici associano un numero intero ad ogni carattere.

Codifiche di caratteri

1968 ASCII.

Codice a 7 bit: 95 caratteri stampabili e 33 di controllo.

1980 Extended ASCII.

Varie estensioni a 8 bit, con simboli grafici e lettere accentate.

1991 Unicode.

Codice a 21 bit (1 milione di simboli). Attualmente (v. 9.0) definiti circa 128.000 caratteri! Viene ulteriormente codificato in **UTF-8**.

1992 UTF-8.

Codifica di Unicode a lunghezza variabile (da 1 a 4 byte). Retro-compatibile con ASCII. UTF-8 è la codifica consigliata per XML e HTML.

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

A: 100 0001

a: 110 0001

"0": 011 0000

Codice ASCII

- Ai primi 32 numerali sono assegnati caratteri di controllo
 - null indica la fine di una stringa
 - carriage return porta il cursore su una nuova riga (andata a capo)
 - horizontal tab è l'usuale carattere tab
- Altro tipo:
 - Bell dovrebbe far suonare un cicalino

Alcuni caratteri Unicode

codice	carattere	significato
U+0434	Д	Lettera cirillica “de”
U+6328	ب	Lettera araba “ba”
U+0E10	ฐ	Lettera thailandese “tho than”
U+1F63A		“Smiling cat face with open mouth”
U+1F382		Torta di compleanno

Il principio di UTF-8

- UTF-8 codifica ciascun carattere Unicode con una sequenza lunga da 1 a 4 byte .Il primo byte di un carattere indica quanto è lunga la sequenza:

Primo byte	Byte totali	Bit a disposizione del carattere
0xxx xxxx	1	7
110x xxxx	2	11
1110 xxxx	3	16
1111 0xxx	4	21

- Tutti i byte successivi nella sequenza hanno il formato 10xx xxxx

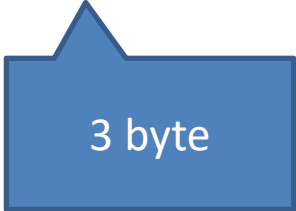
Esempio di UTF-8

- Consideriamo il simbolo dell'euro "€", codice Unicode U+20AC
- E' un codice di 16 bit, quindi richiede 3 byte in UTF-8
- Vediamo come i 16 bit vengono distribuiti su 3 byte da UTF-8:

0x20AC = 0010 0000 1010 1100

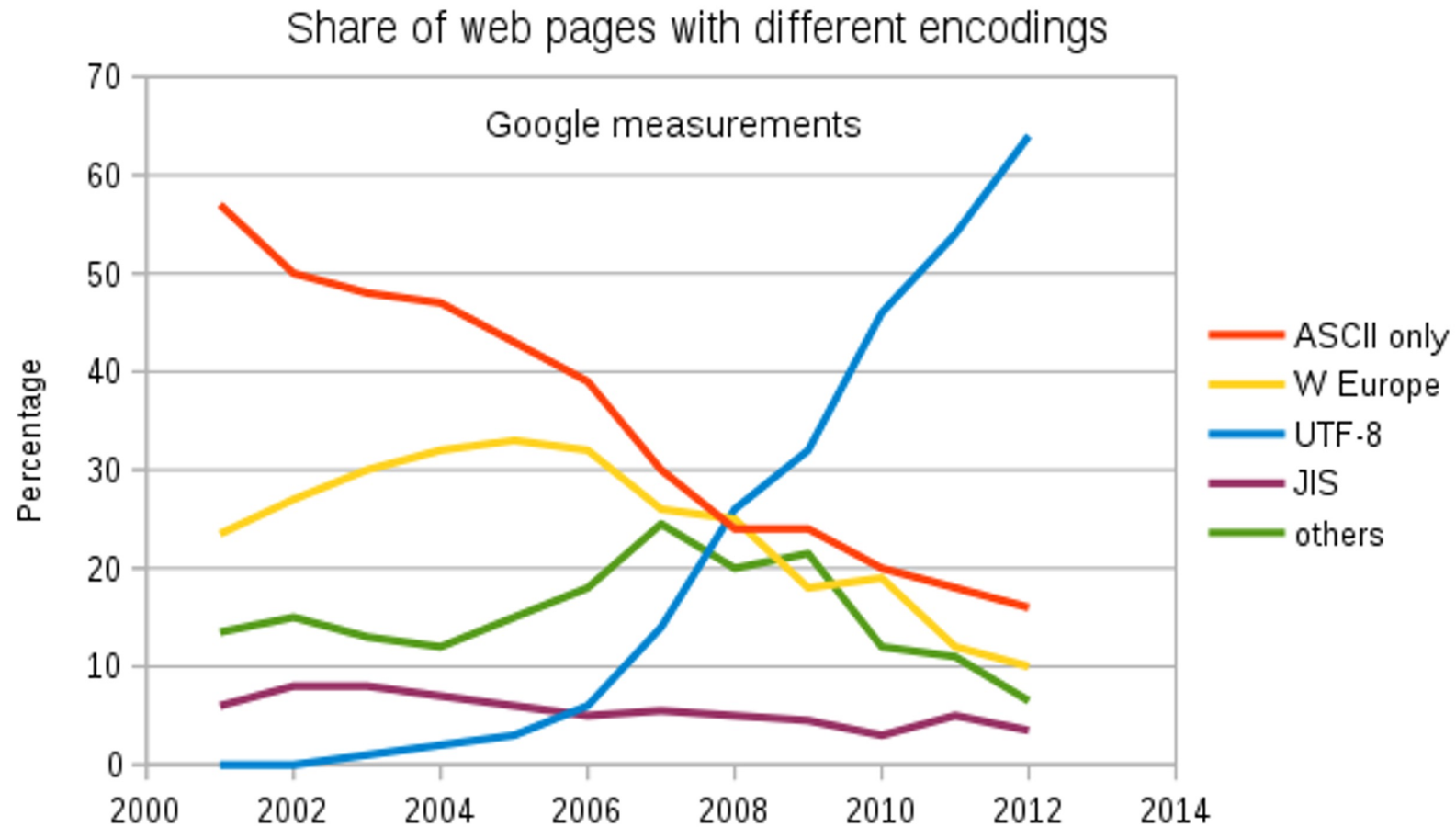
- Codifica UTF-8:

1110 0010 | 1000 0010 | 1010 1100



3 byte

Il successo di UTF-8



- Rappresentare il numero $345,22_6$ in base 5 con una precisione di 3 cifre
- Quanti bit sono necessari e sufficienti per rappresentare il numero 2367 in rappresentazione binaria senza segno? E in complemento a due?
- Rappresentare nello standard IEEE 754 in singola precisione il numero 43,75
- Date parole di 8 bit, si esegua la somma in complemento a due di 56 e -27. Rappresentare il risultato a 8 bit in esadecimale
- Rappresentare il carattere Unicode U+235D in UTF-8

- $1022,143_5$
- $12 \text{ e } 13$
- 0 10000100 010111100000000000000000
- 0x1D
- 1110 0010 1000 1101 1001 1101